PIPELINE DO TRABALHO

Do dado bruto à entrega

O passo a passo do que foi feito, fase por fase — F0 a F8.

Previsão de volume de tráfego em praças de pedágio · Frente 3.2 (redes recorrentes)

Daniel Flavio e Guilherme Martins · Deep Learning · Instituto Federal de Goiás · Outubro de 2026

VISÃO GERAL

O caminho completo, de F0 a F8

F0
Setup
Repositório, ambiente, seed e onde ficam os dados.

F1
Aquisição + auditoria
Baixar os 3 CSVs, auditar e unir em Parquet.

F2
Análise exploratória
Entender os dados e definir o problema e o alvo.

F3
Pré-processamento
Agregar, criar janelas e dividir treino/val/teste.

F4
Baselines + redes
Régua e as três famílias recorrentes treinando.

F5
Optuna
Busca de hiperparâmetros com métrica na validação.

F6
Experimentos
Tudo o que a frente 3.2 exige, com tabelas e gráficos.

F7
Avaliação no teste
Medir uma única vez, no futuro nunca visto.

F8
Entrega
Relatório, slides e revisão final.

Cada etapa produz um artefato e, quando muda uma decisão, entra no `docs/decisoes_dados.md` (D-001 a D-015).

Preparar e conhecer os dados

F0 — Setup (25–26/09)

- Repositório no **GitHub** da dupla
- `.gitignore`: dados e artefatos **fora do Git**
- `requirements.txt`: TensorFlow/Keras, pandas, Optuna
- **Seed global = 42** (reprodutibilidade)
- Dados no **Google Drive** compartilhado

F1 — Aquisição + auditoria (26/09–06/10)

- `download_data.py`: baixa e registra URL, data e **SHA-256**
- Auditar os 3 CSVs revelou **3 surpresas**:
  - encoding **Latin-1** (não UTF-8)
  - `mes_ano` era, na verdade, **diário** (não mensal)
  - `categoria_eixo` com drift + **4.853** chaves repetidas (Q1/2025)
- União dos 3 anos: **9.184.154** linhas
- Salvo em **Parquet**: ~20 MB (contra 854 MB de CSV)

Decisões registradas: D-001 a D-008. Nenhuma surpresa quebraria tudo depois — foi pega na auditoria.

Fonte: ANTT / dados.gov.br

F2 · ANÁLISE EXPLORATÓRIA

Entender os dados e definir o problema

- Período: **2024-01-01 a 2026-08-31** (974 dias)
- **345** praças e **43** concessionárias; **303** com ≥ 180 dias (D-010)
- Alvo: **total de veículos no dia t+1**; unidade = (concessionária, praça, dia) (D-009)
- Distribuição do alvo: **skew 14,9**, cauda longa e **sem zeros**
- Série **descontínua**: só 47 praças cobrem os 974 dias
- Anomalias: critério **IQR por praça** (3× o IQR) e decisão de **manter** — 2,09% dos dias, 2,59% do volume (D-015)
- Os maiores casos são reais: **Reveillon** numa rodovia do litoral
- 4 figuras em `results/figures/` (série, histograma, dia da semana, heatmap)

Decisões **D-009 a D-015**.

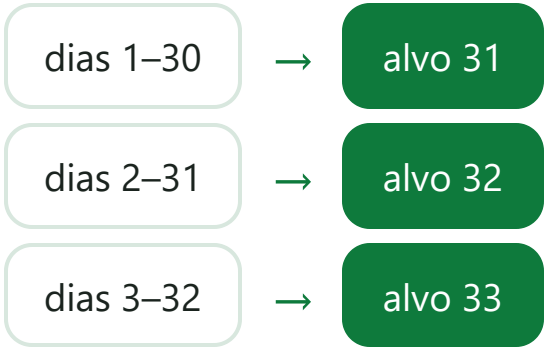
dias anteriores (13 sinais) → **total do dia seguinte** → 1 número (regressão)

F3 · PRÉ-PROCESSAMENTO

Da tabela para a janela da rede

Montagem

- Série `praça × dia` (180.737 linhas)
- **13 sinais**: 4 de volume (Passeio, Comercial, Moto, Total) + 9 de calendário
- **Janelas**: X = janela × 13 → y = Total do dia seguinte
- Janela só vale se **todos os dias existem** (D-011)
- Tipo ausente em dia observado = **0** (D-012)



Divisão e cuidado

- **Split 1 — cronológico**: treino ≤ 2025-11-30 · val até 2026-04-30 · teste depois (D-013)
- **Split 2 — por grupos**: 70/15/15 praças, seed 42
- Padronização ajustada **só no treino** (D-014)
- Testes de **vazamento temporal** passando

A janela **desliza 1 dia** por vez: cada passo = **1 exemplo**.

Nº de janelas = **dias − janela**. Ex.: praça de **180 dias**, janela 30 → **150**.

Total no projeto: **163.899** janelas (105.754 treino + 30.678 val + 27.467 teste).

F4 · MODELAGEM

A régua e as três arquiteturas

Baselines (régua obrigatória)

- **Persistência**: amanhã = hoje
- **Média móvel**: média da janela
- Um modelo só tem valor se **bater essa régua**

Config de referência

- GRU · janela 30 · 1 camada · 64 unidades · Adam
- Early stopping + ReduceLROnPlateau + `clipnorm` + `shuffle=False`

RODADA-PILOTO (MAE NA VALIDAÇÃO)	GRU	LSTM	SIMPLERNN
Erro médio (veículos)	**1.108**	1.138	1.249

Curvas de treino/validação salvas em `results/figures/curva_*.png`: mostram **overfitting leve**, contido pelo early stopping. `src/model.py` + `src/train.py`.

F5 · HIPERPARAMETRIZAÇÃO

Em vez de chutar, buscar com método

- Ferramenta: **Optuna**, métrica = **RMSE na validação** (teste intocado)
- Pruner: **MedianPruner**; storage **SQLite** (`data/optuna_study.db`)
- Espaço: família · janela {7,14,30} · units 32–128 · camadas {1,2} · dropout 0–0,3 · lr · batch · otimizador
- **15 trials** (9 completos) em **13,4 min** — reduzido por causa da CPU

Ressalva honesta

A config **vencedora** da busca, retreinada com os dados completos, rendeu **pior** (MAE ≥ 1.493) que a referência (1.108). Por isso o **modelo final usa a referência**.

Isso mostra, com número, que **o orçamento da busca foi pequeno** (sem GPU).

Registro em `results/optuna_trials.csv`, `optuna_resumo.json` e `config_final.json`.

F6 · EXPERIMENTOS DA FRENTE 3.2

Tudo o que o enunciado pede, comparado

EXPERIMENTO	RESULTADO (VALIDAÇÃO)
Famílias (mesma receita)	GRU **1.537** · LSTM 1.550 · SimpleRNN 1.566 (MAE)
Janelas de tempo	**30 dias** melhor (RMSE 2.725 vs 3.020 / 3.061)
Profundidade	2 camadas > 1 (na receita controlada); a 3ª não acrescenta
Largura	128 unidades melhora o erro, mas com **12× mais parâmetros** para ~12% de MAE
Custo	nº de parâmetros, tempo de treino e **ms por previsão**

Dependências de longo prazo: janelas maiores ajudam. Estratégia: **1 passo à frente** (multi-step não implementado — limitação declarada). Figuras: `exp_familia`, `exp_janela`, `exp_largura`, `exp_custo_parametros`.

F7 · AVALIAÇÃO FINAL

O teste usado uma única vez

Continuar a série da mesma praça

MODELO	MAE	R^2
Repetir o último dia	1.536	0,92
Média dos últimos dias	1.831	0,88
GRU (final)	**1.126**	**0,95**

Prever em praças novas

MODELO	MAE	R^2
Repetir o último dia	1.774	0,92
Média dos últimos dias	2.028	0,89
GRU (final)	**1.469**	**0,94**

O modelo bate a régua nos dois cenários: **~27%** menor erro no primeiro e **~17%** no segundo. Como escala, o erro de 1.126 equivale a **~8%** do movimento diário médio.

F8 · ENTREGA

Relatório, slides e revisão final

O que foi produzido

- `report/relatorio.md` — as 6 seções do PDF
- `slides/apresentacao_v2.pdf` — 14 slides, 16:9 (hand-drawn)
- Validação **requisito a requisito** do enunciado

O que ainda falta

- **Pendente** Ensaiar e cronometrar a apresentação (**15 min**)
- **Pendente** Deixar o **link do GitHub público** para o professor

Praticamente todos os requisitos formais estão atendidos. As duas pendências acima dependem só da dupla.

REPRODUTIBILIDADE

Artefatos e como rodar do zero

Artefatos

- `src/` : preprocess · dataset · model · train · optuna_search · experimentos · avaliar_teste
- `results/` : tabelas (CSV/JSON) e figuras (PNG)
- `docs/decisoes_dados.md` : **D-001 a D-015**
- `report/` e `slides/`

Ordem dos comandos

```
python download_data.py
python src/preprocess.py
python src/eda.py
python src/dataset.py
python src/train.py
python src/optuna_search.py
python src/experimentos.py
python src/avaliar_teste.py
```

Os dados ficam fora do Git; o download e o manifesto com **SHA-256** garantem a reprodutibilidade.

PONTO DE SITUAÇÃO

Aprendizado e limites

O que funcionou

As redes recorrentes superaram a régua nos dois cenários. A diferença entre as três famílias é **pequena** — o SimpleRNN entrega quase o mesmo com **um terço dos parâmetros**.

Limitações

Sem GPU, a busca foi reduzida. Não houve validação cruzada. Só prevemos **1 dia à frente**. Dias atípicos (feriados) são os mais difíceis.

A lição central: o maior ganho veio de **decisões sobre os dados** (auditoria, granularidade diária, janela de 30 dias) — não da escolha da célula recorrente.